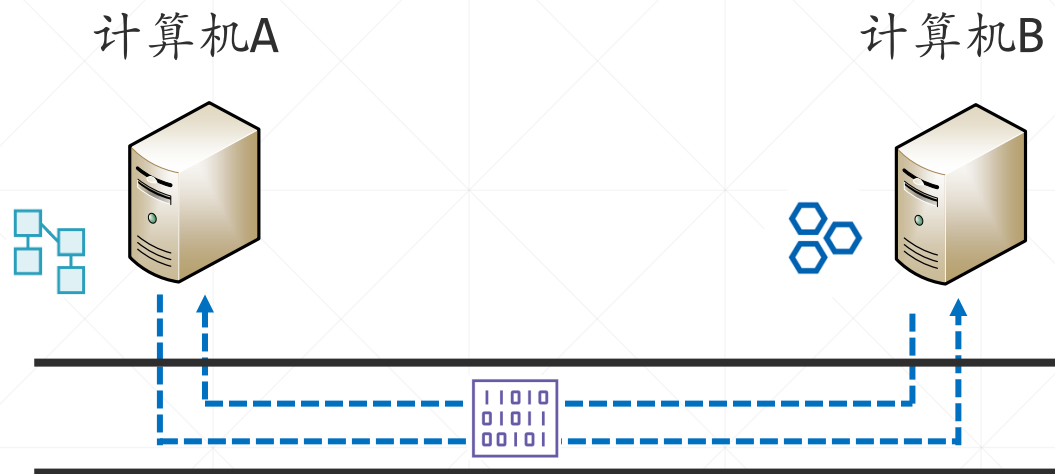


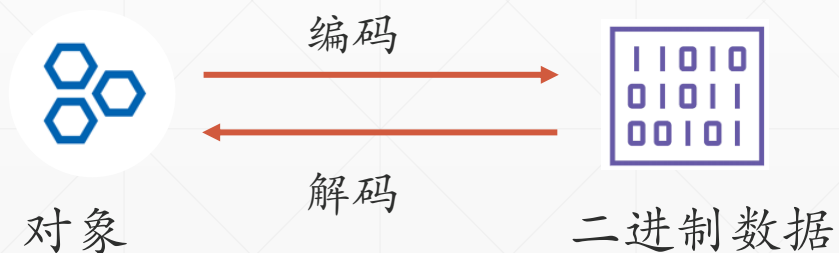
Netty自定义编码器与解码器

北京理工大学计算机学院
金旭亮

编码与解码概述



在计算机网络中，两台计算机之间通过网络传输的数据，都是二进制形式的，但跑在计算机里的网络应用，通常都使用对象来表达信息，这就存在着一个在“对象”与“二进制”数据相互转换的问题。



在Netty应用中，二进制数据是放到ByteBuf中的，然后再从ByteBuf中读取数据，创建消息对象，这个就是“解码”，反过来，将对象转换为二进制数据，再写入ByteBuf中，称为“编码”。

Netty应用通讯数据的编码与解码



当通过Netty发送或者接收一个消息的时候，就将会发生一次数据转换。



入站消息会被解码；也就是说，从字节转换为另一种格式，通常是一个Java对象，解码器类名，通常有“ByteToMessageDecoder”后缀。



如果是出站消息，则会发生相反方向的转换：它将从它的当前格式被编码为字节，编码器类名，通常有“MessageToByteEncode”后缀。

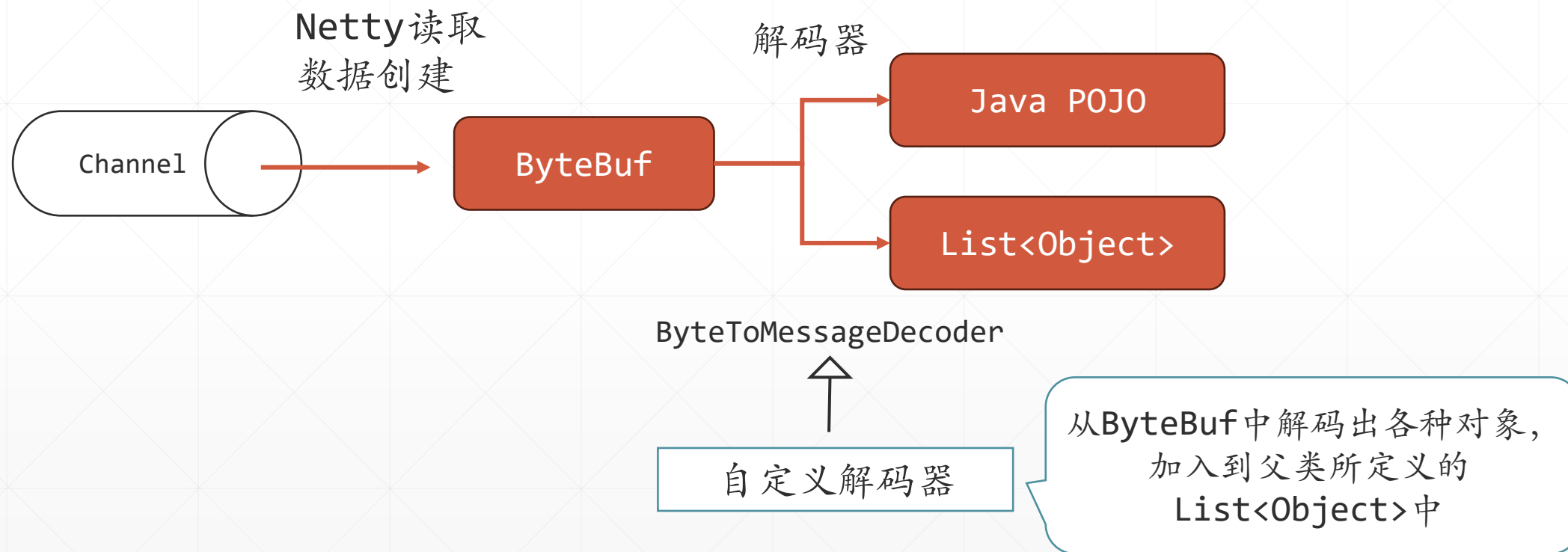


这两种方向的转换的原因很简单：网络数据总是一系列的字节。



编解码，对应于序列化和反序列化，完成之后，Netty会自动地调用`ReferenceUtil.release()`方法释放。

Netty的数据解码过程

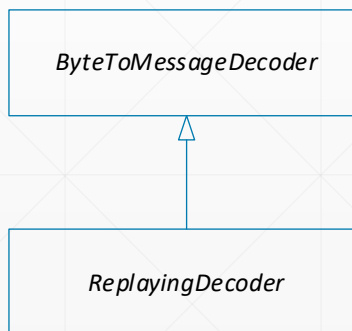


解码器示例

这个解码器，从ByteBuffer中读入4个字节，将其“拼”成一个int整数。

```
public class ByteToIntegerDecoder extends ByteToMessageDecoder {  
    @Override  
    protected void decode(ChannelHandlerContext channelHandlerContext,  
                           ByteBuffer byteBuf, List<Object> list) throws Exception {  
        if (byteBuf.readableBytes() >= 4) {  
            int num = byteBuf.readInt();  
            list.add(num);  
            System.out.println("ByteToIntegerDecoder接收到整数: "+num);  
        }  
    }  
}
```

注意在解码前，需要验证输入的ByteBuffer中已有足够的数据，如果派生自ReplayingDecoder，则它会自动完成上述任务。



编码器示例

解码的反过程，就是编码。在Netty中，消息要经过编码为二进制数据，写入到ByteBuf中，才能够被发送出去。编码器的基类，是MessageToByteEncoder。

```
public class IntegerToByteEncoder extends MessageToByteEncoder<Integer> {  
    @Override  
    protected void encode(ChannelHandlerContext channelHandlerContext,  
                           Integer integer,  
                           ByteBuf byteBuf) throws Exception {  
        byteBuf.writeInt(integer);  
        System.out.println("IntegerToByteEncoder: Integer "+integer+" 被写入到ByteBuf中");  
    }  
}
```

上述编码器示例，将一个整数，拆成四个字节，再写入到ByteBuf中。

注意其泛型参数设定为Integer，这就意味着，它只能接收Integer类型的数据，而会“拒绝”所有其他类型的数据。

不同消息类型之间的转换

Netty网络应用实现的功能，通常是由一个包容有多个“通道处理程序（ChannelHandler）”的“流水线（Channel Pipeline）”实现的。

每个处理程序，往往需要接收特定的类型的消息，加工与处理之间，转换为另一种格式的消息，再转发给流水线中下一个处理程序。

为此，Netty定义了一个MessageToMessageDecoder<T>，用于完成这个任务，以下是一个示例，这个编码器，把一个Integer对象，转换为一个字符串：

```
public class IntegerToStringDecoder extends MessageToMessageDecoder<Integer> {  
    @Override  
    protected void decode(ChannelHandlerContext ctx, Integer msg, List<Object> out) {  
        String numStr = String.valueOf(msg);  
        System.out.println("IntegerToStringDecoder收到整数: " + msg + " 转换为"  
            + numStr.getClass().getSimpleName() + ":" + numStr);  
        out.add(numStr);  
    }  
}
```

最后，写一个“业务类型”处理器

```
public class MyInboundHandler extends ChannelInboundHandlerAdapter {
    @Override
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws Exception {
        var info = "MyInboundHandler收到的数据: " + msg;
        System.out.println(info);
        var classInfo = "数据类型为: " + msg.getClass().getSimpleName();
        System.out.println(classInfo);
        //这里假设进行业务处理，比如汇总计算订单金额之类，得到结果为1000
        //将结果发回给客户端
        System.out.println("MyInboundHandler业务处理结束，向管线中写入数据:1000");
        ctx.writeAndFlush(1000);
    }
}
```

这个处理器，计划放在管线的末端，完成业务处理后，将结果传回给客户端，这个结果，将会被送给出站的“编码器”，本例中是IntegerToByteEncoder。

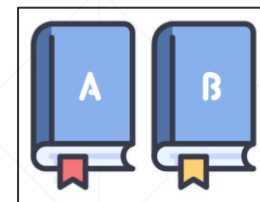
测试自定义编解码器

```
@Test
public void testEncoderDecoder() {
    //构建数据处理流水线
    ChannelInitializer initializer = new ChannelInitializer<EmbeddedChannel>() {
        @Override
        protected void initChannel(EmbeddedChannel ch) throws Exception {
            ch.pipeline().addLast(new IntegerToByteEncoder());
            ch.pipeline().addLast(new ByteToIntegerDecoder());
            ch.pipeline().addLast(new IntegerToStringDecoder());
            ch.pipeline().addLast(new MyInboundHandler());
        }
    };
    var channel = new EmbeddedChannel(initializer);
    ByteBuf buf= Unpooled.buffer();
    buf.writeInt(1);
    //模拟客户端将整数1发给服务端，这将启动整条流水线
    channel.writeInbound(buf);
    //读取最终的结果
    var result = channel.readOutbound();
    //显示将要发送到网络上的最终的ByteBuf的内容
    System.out.println(ByteBufUtil.prettyHexDump((ByteBuf) result));
    //关闭退出
    channel.finish();
    channel.close();
}
```

测试结果

```
Run  EncoderDecoderTests.testEncoderDecoder x
[Icons]
✓ EncoderDecoderTests 141 ms  ✓ Tests passed: 1 of 1 test - 141 ms
  ✓ testEncoderDecode 141 ms
    "C:\Program Files\Java\jdk-21\bin\java.exe" ...
    ByteToIntegerDecoder接收到整数: 1
    IntegerToStringDecoder收到整数: 1 转换为String:1
    MyInboundHandler收到的数据: 1
    数据类型为: String
    MyInboundHandler业务处理结束, 向管线中写入数据:1000
    IntegerToByteEncoder: Integer 1000 被写入到ByteBuf中
      +-----+
      | 0 1 2 3 4 5 6 7 8 9 a b c d e f |
      +-----+
      |00000000| 00 00 03 e8 |....|
      +-----+
    Process finished with exit code 0
```

最终ByteBuf中存储的内容，
是十六进制表示的3e8，换算
为10进制，正好就是1000。



编码器与解码器可以合并，为此，Netty提供了ByteToMessageCoder和MessageToMessageCoder两个基类，可以把encode()和decode()合并在一个类中，以避免写两句代码分别将这些Handler加入到流水线中。



如果不想合并编码器与解码器的代码，则可以采用CombinedChannelDuplexHandler<Decoder, Encoder>，直接把两个现有的编码器和解码器组合在一起。